

So sánh Hiệu năng Baseline, SARIMAX và LSTM trong Dự báo Nhu cầu Bán lẻ

Hướng dẫn Project 2 — Mạnh

1. BỐI CẢNH

- Đề tài tổng thể: so sánh các phương pháp dự báo nhu cầu bán lẻ trên dataset M5 (Walmart/Kaggle)
- Nghiên cứu so sánh 5 models: Naive, Seasonal Naive, SARIMAX, Prophet, Global LSTM
- Em chịu trách nhiệm **implement 4 models** (trừ Prophet — anh tự làm), **evaluation module**, và **chạy experiments**
- Có 2 phases: Phase 1 (basic features — so sánh công bằng), Phase 2 (thêm features nâng cao — đánh giá thực tế)

2. DATASET & INPUT

- **M5 Forecasting Competition:** <https://www.kaggle.com/competitions/m5-forecasting-accuracy> — daily sales, item-store level
- Em sẽ **nhận sẵn** data đã clean (không cần tự xử lý): sales_clean.parquet (daily sales + features), selected_series.csv (100 cặp item-store), split_config.json (train/test config)
- **Evaluation:** expanding window, 7-8 folds, horizon 14 ngày, test period 2016-03-01 → 2016-06-19

3. DELIVERABLES

3.1 Naive + Seasonal Naive

- src/models/naive.py
- **Naive:** predict = giá trị cuối cùng của training data, lặp lại 14 ngày. Tức $y_{pred}[t] = y_{train}[-1]$ cho mọi t trong horizon.
- **SNaive:** lặp lại 7 ngày cuối cùng. Tức $y_{pred}[t] = y_{train}[-(7 - t\%7)]$ — lặp pattern tuần cuối.

- Không dùng features, chỉ dùng sales history
- **Baseline là bắt buộc** — nếu LSTM không thắng Naive thì đó cũng là finding có giá trị

3.2 SARIMAX

- src/models/sarimax_model.py — dùng pmdarima (auto_arima), seasonal m=7
- auto_arima config: max_p=3, max_q=3, max_P=1, max_Q=1, seasonal=True, m=7, stepwise=True, suppress_warnings=True
- Timeout mỗi series/fold = 60 giây — nếu quá thời gian → coi như fail
- **Phase 1 exogenous:** day_of_week, month, is_holiday, is_weekend
- **Phase 2 thêm:** rolling_7, rolling_28, lag_7, lag_14, lag_28, sell_price, snap

Cách xử lý exogenous cho forecast horizon:

- Các features `day_of_week`, `month`, `is_holiday`, `is_weekend`, `snap` là **deterministic** — biết trước cho tương lai vì chúng lấy từ calendar
- Khi predict: lấy exog cho 14 ngày forecast từ sales_clean.parquet (các cột calendar đã có sẵn cho test dates)
- **Phase 2:** `rolling_7`, `rolling_28`, `lag_7`, `lag_14`, `lag_28` **cũng đã được precompute** (shift(1)) trong sales_clean → lấy trực tiếp cho test dates. `sell_price` cũng lấy từ data.
- Tóm lại: filter sales_clean cho test dates → lấy cột exog cần thiết → truyền vào
`model.predict(n_periods=14, exogenous=future_exog)`

Fallback rule: khi SARIMAX không converge hoặc timeout → dùng Seasonal Naive cho series/fold đó → ghi `fallback=True` trong predictions, `converged=False` trong training_log. Không giấu fallback — ghi rõ trong report số series/folds bị fallback.

3.3 Global LSTM

- src/models/lstm_model.py — PyTorch, **1 model duy nhất** cho tất cả 100 series (Global, không Local)
- Config: hidden=64, layers=1, dropout=0.2, lookback=28, output=14, epochs=50, early stopping patience=5, Adam lr=0.001, MSE loss, batch=64

Phase 1 input features (5): sales, day_of_week, month, is_holiday, is_weekend **Phase 2 thêm (7):** rolling_7, rolling_28, lag_7, lag_14, lag_28, sell_price, snap

Cách tạo sequences (QUAN TRỌNG):

- Với mỗi series, tạo sliding windows: input = 28 ngày liên tiếp (tất cả features), output = 14 ngày tiếp theo (chỉ sales)
- Input shape: (batch_size, lookback=28, n_features) — ví dụ Phase 1: (batch, 28, 5)
- Output shape: (batch_size, horizon=14) — chỉ predict sales
- Stride = 1 (trượt 1 ngày mỗi window)
- **Gộp sequences từ tất cả 100 series** vào cùng 1 training set → shuffle → train Global model
- **Yêu cầu tối thiểu:** mỗi series phải có ít nhất `lookback + horizon = 42` ngày training data trong fold đó. Nếu không đủ → **bỏ qua series đó cho fold đó**, ghi log `error_msg = "insufficient_training_data"`

Normalization (CHỐNG LEAKAGE):

- `MinMaxScaler` fit **CHỈ trên training data** của fold hiện tại, per feature (tức fit trên tất cả 100 series training data gộp lại)
- Transform cả train và test data bằng scaler đã fit
- Sau predict → **inverse transform** để ra scale gốc
- **Clamp output ≥ 0** (sales không thể âm): `y_pred = np.maximum(y_pred, 0)`

Anh sẽ hỗ trợ 2-3 buổi pair-code cho phần setup PyTorch, training loop, normalization

3.4 Evaluation module

- `src/evaluation/metrics.py` — MAE, RMSE, MASE, MAPE
- Đây là **single source of truth** — tất cả models (kể cả Prophet do anh chạy) dùng cùng module này

Công thức chi tiết:

Metric	Công thức	Ghi chú
MAE	<code>`mean(</code>	<code>y_true - y_pred</code>
RMSE	<code>sqrt(mean((y_true - y_pred)²))</code>	Phạt errors lớn nặng hơn
MASE	<code>MAE_model / MAE_snaive_1step</code>	Xem chi tiết bên dưới
MAPE	<code>`mean(</code>	<code>y_true - y_pred</code>

MASE chi tiết:

- MAE_snaive_1step = lỗi trung bình của Seasonal Naive **1-bước** trên **training data**
- Công thức: $MAE_snaive_1step = \text{mean}(|y_train[t] - y_train[t-7]|)$ cho $t = 7, 8, \dots, \text{len}(\text{train})-1$
- $MASE < 1 \rightarrow$ model tốt hơn baseline. $MASE > 1 \rightarrow$ model tệ hơn baseline.
- **Edge case:** nếu $MAE_snaive_1step \approx 0$ (series gần constant) $\rightarrow MASE = \text{NaN}$

MAPE rule: tính trên **toàn bộ 14 ngày** của 1 horizon. Nếu **bất kỳ** y_true nào < 1.0 trong horizon đó $\rightarrow MAPE = \text{NaN}$ cho horizon đó. Supplementary only, **KHÔNG** dùng làm metric chính.

Function signature gợi ý:

```
def compute_metrics(y_true: np.ndarray, y_pred: np.ndarray,
                    y_train: np.ndarray, season: int = 7) -> dict:
    """
    Args:
        y_true: actual values, shape (horizon,)
        y_pred: predicted values, shape (horizon,)
        y_train: training data, shape (n_train,) - dùng cho MASE
        season: seasonal period (7 cho weekly)
    Returns:
        {"mae": float, "rmse": float, "mase": float, "mape": float|NaN}
    """
```

3.5 Resource profiler

- `src/evaluation/profiler.py` — đo training time, RAM usage
- Dùng `time.time()` cho `train_time`, `psutil.Process().memory_info().rss / 1024**2` cho `peak_ram_mb`
- `cpu_percent: psutil.cpu_percent(interval=None)` snapshot

3.6 Experiments

- Phase 1: 4 models (Naive, SNaive, SARIMAX, LSTM) $\times$ 100 series $\times$ 7-8 folds $\rightarrow$ `results/*/phase1/`
 - Prophet Phase 1 do anh chạy riêng $\rightarrow$ anh tự save vào `results/prophet/phase1/`
- Phase 2: SARIMAX + LSTM với enhanced features $\rightarrow$ `results/*/phase2/`
 - Prophet Phase 2 cũng anh chạy riêng

Ai tạo metrics_comparison.csv:

- Em tạo metrics cho 4 models của em: save từng model riêng vào `results/{model}/phase{N}/metrics.csv`
- Cuối cùng **em merge** tất cả metrics của 4 models em + anh sẽ giao metrics Prophet cho em → em gộp thành **results/metrics_comparison.csv** (1 file duy nhất, 5 models × 2 phases)
- Schema: model, phase, item_id, store_id, fold, mae, rmse, mase, mape

3.7 Automation + comparison

- `scripts/run_experiments.sh` — 1 lệnh chạy toàn bộ 4 models (Naive, SNaive, SARIMAX, LSTM) cho cả Phase 1 và Phase 2
- `notebooks/03_comparison.ipynb` — tables, box plots so sánh 5 models (sau khi nhận Prophet metrics từ anh)

3.8 Report

4. OUTPUT FORMAT (CẦN ĐÚNG SCHEMA)

predictions.csv — mỗi model, mỗi phase: `results/{model_name}/phase{1|2}/predictions.csv`

Column	Type	Mô tả
date	str (YYYY-MM-DD)	Ngày forecast
item_id	str	Mã sản phẩm
store_id	str	Mã cửa hàng
y_true	float	Giá trị thực
y_pred	float	Giá trị dự báo (≥ 0)
fold	int	Fold number (1-based)
fallback	bool	True nếu SARIMAX fail → dùng SNaive. False cho Naive/SNaive/LSTM.

training_log.csv — mỗi model, mỗi phase:
`results/{model_name}/phase{1|2}/training_log.csv`

Column	Type	Mô tả
--------	------	-------

fold	int	Fold number
item_id	str	Mã sản phẩm
store_id	str	Mã cửa hàng
train_time_s	float	Thời gian train (giây)
peak_ram_mb	float	RAM peak (MB)
cpu_percent	float	CPU usage (%)
converged	bool	True nếu model converge. LSTM: True nếu train xong.
error_msg	str	"" nếu OK, mô tả lỗi nếu fail (ví dụ: "timeout", "insufficient_training_data")

Lưu ý training_log cho LSTM:

- Vì Global LSTM train 1 model cho tất cả series trong 1 fold → `train_time_s` giống nhau cho tất cả series cùng fold. Ghi 1 giá trị, lặp lại cho 100 rows.
- `converged = True` nếu training hoàn tất (dù early stopping). `False` chỉ khi crash/error.

metrics_comparison.csv — results/metrics_comparison.csv (merged tất cả models)

Column	Type	Mô tả
model	str	"naive", "snaive", "sarimax", "prophet", "lstm"
phase	int	1 hoặc 2
item_id	str	Mã sản phẩm
store_id	str	Mã cửa hàng
fold	int	Fold number
mae	float	Mean Absolute Error
rmse	float	Root Mean Squared Error
mase	float	Mean Absolute Scaled Error
mape	float	Mean Absolute Percentage Error (NaN nếu $y_{true} < 1.0$)

5. CÁC BƯỚC THỰC HIỆN

Bước 1 — Tìm hiểu lý thuyết (tuần 1-2)

- Prerequisite: hiểu cơ bản time series (stationarity, ACF/PACF, differencing)
- Đọc ARIMA theory (FPP3 Ch.9), LSTM (Colah's blog), evaluation metrics (FPP3 Ch.5)

Bước 2 — Implement Baselines (tuần 2)

- Code Naive + Seasonal Naive, test trên 1-2 series để verify output format
- Phần đơn giản nhất — nên hoàn thành sớm
- Verify: predictions.csv đúng schema, $y_{\text{pred}} \geq 0$

Bước 3 — Implement SARIMAX (tuần 3-4)

- Cài pmdarima, implement wrapper với auto_arima (config như mục 3.2)
- Implement timeout 60 giây mỗi series/fold (dùng `signal.alarm` trên Linux hoặc `threading.Timer`)
- Test trên 1 series: verify convergence, output format
- Implement fallback logic (SNaive khi SARIMAX fail hoặc timeout), test fallback
- **Test exogenous handling:** verify future exog lấy đúng từ sales_clean cho test dates

Bước 4 — Implement Evaluation + Profiler (tuần 4)

- Code metrics.py với 4 metrics theo công thức mục 3.4
- Code profiler.py
- **Test metrics trên known values** (tính tay → so sánh với code):
 - Ví dụ: $y_{\text{true}} = [3, 5, 2]$, $y_{\text{pred}} = [4, 4, 3]$ → $\text{MAE} = 1.0$, $\text{RMSE} \approx 1.0$
 - MASE: tự tạo y_{train} giả → tính $\text{MAE}_{\text{snaive_1step}}$ tay → verify

Bước 5 — Implement Global LSTM (tuần 4-6)

- Prerequisite: pair-code với anh để setup PyTorch + training loop
- Code GlobalLSTM model class, data preparation (sequences, normalization)
- Code training loop (with early stopping)
- Test trên 5 series: verify training works, output format đúng, $y_{\text{pred}} \geq 0$

Bước 6 — Chạy Experiments (tuần 6-8)

- Phase 1: 4 models × 100 series. Phase 2: SARIMAX + LSTM enhanced

- Save predictions + training logs per model/phase
- Tính metrics per model/phase → save per-model metrics

Bước 7 — Merge Metrics + Automation + Comparison (tuần 8-9)

- Nhận Prophet metrics từ anh → merge thành metrics_comparison.csv
- Viết run_experiments.sh, tạo comparison notebook

6. TÀI LIỆU THAM KHẢO

- Hyndman & Athanasopoulos (2021). *FPP3*, Ch.5, Ch.9 — <https://otexts.com/fpp3/>
- Colah's Blog — "Understanding LSTM Networks"
- Hyndman & Koehler (2006). "Another look at measures of forecast accuracy." *IJF* 22(4)
- Bandara et al. (2020). "Forecasting across time series databases using RNNs." *Expert Sys. App.* 140
- Makridakis et al. (2022). "M5 accuracy competition: Results." *IJF* 38(4)

7. GỢI Ý CẤU TRÚC REPORT

1. Giới thiệu — bối cảnh demand forecasting, tại sao cần so sánh models
2. Literature Review — tổng quan ARIMA, LSTM trong forecasting
3. Phương pháp — dataset, 4 models, evaluation protocol, 2-phase design
4. Kết quả — Phase 1, Phase 2, computational cost, SARIMAX convergence rate
5. Discussion — model nào tốt nhất, features có giúp không, accuracy vs complexity
6. Kết luận

8. LƯU Ý

- Output cần đúng schema (column names, types) như mục 4 — để đảm bảo tích hợp được với các phần khác
- SARIMAX fallback phải transparent — ghi rõ số series/folds fail trong report
- LSTM y_{pred} phải ≥ 0 (clamp sau inverse transform)
- Nếu em muốn thử thêm approach khác cho LSTM (architecture, features) thì trao đổi với anh
- Mỗi function nên có docstring giải thích input/output

